

SPRING MVC

199



What is Spring MVC?

1. A Spring MVC is sub framework of Spring Framework which is used to build a **web application**.
2. It is build on the top of **Servlet API**.
3. It follows the **Model-View-Controller** design pattern.
4. It implements all the basic features of a core spring framework like **Inversion of Control, Dependency Injection**.



Learn with Durgasir

Simplified:

- **Model** = Data
- **Controller** = Logic
- **View** = User Display (Format)

How does HTTP req or Browser/Server communicate?

Summary:

- **How does it travel?:** As an **HTTP request** over the internet.
- **What form?:** The request contains information like the URL, request method (GET, POST), headers, and possibly some data.
- **Flow:** Browser → HTTP request → Server → Servlet → Processed data → HTTP response → Browser displays page.

Servlets -> Server Side java Programs, Servlets are responsible for handling incoming requests, processing them, and sending back a response. They are the backbone of handling requests in early Java web applications

LifeCycle of an HTTP Request? How MVC Pattern Works ?

Front Controller Pattern in the Diagram:

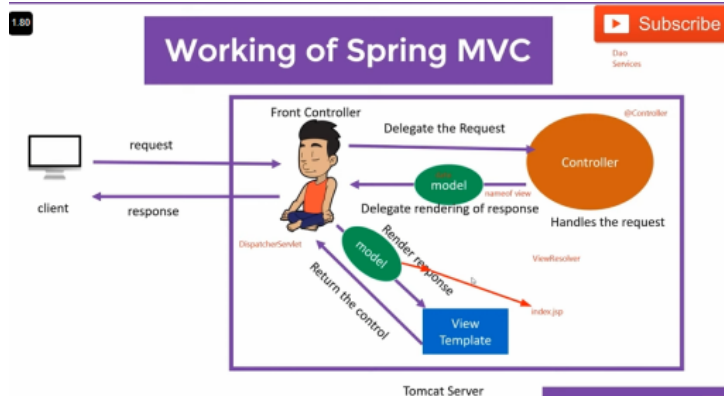
- **Web Browser:** Sends a request (e.g., accessing a page).
- **Front Controller (DispatcherServlet):** Receives the request first, then delegates it to the specific controller.
- **Controller:** The controller processes the request, interacts with the model, and prepares the response.
- **View:** The controller sends the processed data to the view, which renders the HTML or other response formats to the user.

Why Spring MVC ?

1. Separate each role modal , view , controller etc
2. Powerful configuration.
3. It is sub framework of Spring Framework. Use of Spring core features like IOC etc.
4. Rapid Application Development.
5. Spring MVC is Flexible , easy to test and much features.



Working of Spring MVC ?



Difference between Traditionall and Spring MVC ?

Aspect	Without Spring MVC (Traditional Servlet/JSP)	With Spring MVC
Request Handling	Requests are directly handled by individual Servlets .	DispatcherServlet (Front Controller) intercepts all requests and forwards them to the appropriate Controller .
URL Mapping	Mapped manually in <code>web.xml</code> or annotations on individual Servlets.	Managed by Spring annotations like <code>@RequestMapping</code> or <code>@GetMapping</code> .
Business Logic	Business logic and request handling are done inside Servlets .	Controllers handle request logic and delegate to services or models for business processing.
View Handling	Servlet uses <code>RequestDispatcher</code> to forward to a JSP.	Controller returns the view name (e.g., Thymeleaf or JSP), and Spring resolves it automatically.
Separation of Concerns	Mixed responsibilities: Servlet handles both request processing and logic.	Clear separation: Controller handles logic; View (Thymeleaf/JSP) generates the HTML.
Simplified Code	Requires more boilerplate code to handle requests, forward data, etc.	Spring MVC abstracts most of the work (like request forwarding), so you write less code.

Download and Configure Tomcat in IntelliJ or use SmartTomcat Plugin
Add Spring MVC dependency and Jakarta - servlet in Maven, it will automatically include all Spring core and other important dependencies.

Steps to Create MVC Web App.

1. Configure the **dispatcher servlet** in web.xml

2. Create **Spring Configuration File**

3. Configure **View Resolver**

4. Create **Controller**

5. Create a View to show (page)



1) Config. Front Controller (Dispatcher Servlet)

Configure the web.xml file

Download web_app/servlet 4 schema xsd from this link ..

<http://www.oracle.com/webfolder/technetwork/jsc/xml/ns/javaee/index.html>

and then set web.xml as

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">

  <!-- Configure DispatcherServlet -->
  <servlet>

    </servlet>
</web-app>
```

All the servlets or server side Java code is created in Main Folder, whereas all the views and JSP files are created in WEB-APP Folder

1) Servlet Mapping in web.xml

```

<!-- Declaration of Servlet -->
<servlet>
    <servlet-name>
        spring
    </servlet-name>
    <servlet-class>
        org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
</servlet>

<!-- Mapping of Servlet -->

    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>

```

For Dispatcher Servlet (Front Controller) Mapping use / as the url-pattern , and for other servlet , you can use /anything

2) Configure Spring.xml file

Create a .xml file in WEB-INF Folder , with the name as DispatcherServletName-servlet.xml // IMPORTANT

for example if your dispatcher servlet is name sky then your spring configure file should be named as sky-servlet.xml

```

<?xml version="1.0" encoding="UTF-8" ?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

</beans>

```

3) Create ViewResolver bean

In simple terms:

- When your controller returns a **view name** like "home", this resolver will look for a file in this path:
 - /WEB-INF/views/home.jsp
- The **prefix** and **suffix** just help define the folder (WEB-INF/views/) and file extension (.jsp) for all your view files.

So, instead of writing the full path for every view, you just return "home" or "about" from your controller, and Spring will automatically find /WEB-INF/views/home.jsp or /WEB-INF/views/about.jsp.



```
<bean name="viewResolver" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/views/" />
  <property name="suffix" value=".jsp" />
</bean>
```

4) Create Controller

```

5
6  @Controller no usages
7  public class HomeController {
8
9      @RequestMapping("/home") no usages
10     public String home(){
11         return "index";
12     }
13
14 }
15
```

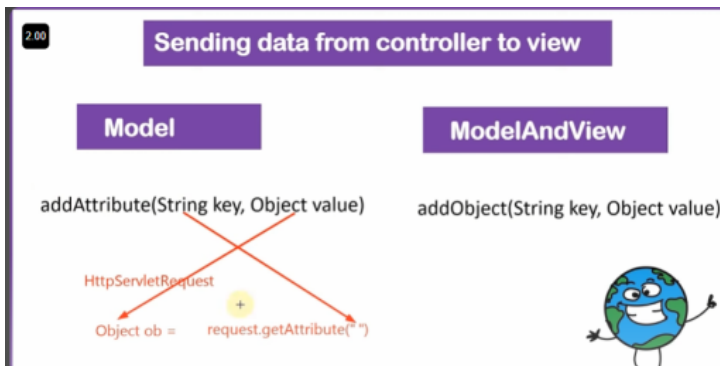
@Controller - to define a class as a controller

@RequestMapping("/url") - over a function that returns the name of the jsp file also use

`<context:component-scan base-package="Controller"/>`

IMP : Check the version compatibility of spring-mvc , servlet-api and tomcat

Passing Data from Controller -> View



Model

We have to pass the Object of Model or ModelAndView in the controller function and use the `model.addAttribute()` to set value , and on the view (.jsp) side , we have to use `request.getAttribute("key")` to get the values.

ModelAndView

This is a bit different from model cause here we have to create and return an object of ModelAndView in the controller function , and have to add the attributes and view name also in between

```

@RequestMapping("/help") no usages new *
public ModelAndView help() {

    ModelAndView md = new ModelAndView();
    md.addObject( attributeName: "name", attributeValue: "Rjt");
    md.setViewName("help");
    LocalDate date = LocalDate.now();
    md.addObject( attributeName: "date",date);

    return md;
}

```

@RequestMapping()

```

@RequestMapping(path="/about",method= RequestMethod.POST) no
public String about(){

    System.out.println("This is Home Controller Servlet");
    return "about";
}

```

Create .war file and Deploy it on Tomcat

